

LESTAT

Testez les mots de passe de vos utilisateurs
avant qu'un pirate vous fasse un audit gratuit

INTRODUCTION

Nous allons parler ici de cassage/craquage de mots de passe.

Et de comment faire, en tant que défenseur, pour ne pas se faire surprendre.

À propos de l’auteur :

David SORIA (@Sibwara) pentester depuis 2011

Fondateur de Astar

CASSER DES HASH

PETIT RAPPEL POUR LES MOLDUS

Les serveurs ne stockent pas vraiment votre mot de passe.

Ce serait trop risqué : s'ils se font pirater, tous les mots de passe des utilisateurs fuiteraient directement

À la place, ils stockent **le résultat que produit votre mot de passe** quand on le met dans une fonction (appellée fonction de «hachage»)

Comme ça, quand vous voulez vous connecter, il n'y a qu'à mettre le mot de passe reçu dans la même fonction et vérifier que ça donne le même résultat (appelé «hash») que celui qu'on avait enregistré

Ex : F(«Password») = 8be3c943b1609fffbfc51aad666d0a04adf83c9d

PETIT RAPPEL POUR LES MOLDUS

Donc quand on parle de **cassage de mots de passe**, il y a souvent deux réalités distinctes :

- le cassage «en ligne» où on envoie des mots de passe au serveur jusqu'à ce qu'il répondre «c'est bon»
  (environ 1 000 essais par seconde)
- le cassage «hors ligne» où on a récupéré un hash et on fait passer des mots de passe dans la fonction de hachage jusqu'à trouver celui qui produit le même résultat
  (environ 1 000 000 000 essais par seconde)

«Casser» c'est quand c'est gagné !

PETITE RETROSPECTIVE

Pendant des années, les pirates utilisaient du «bruteforce» pour casser les mots de passe : a, b, c, ..., z, aa, ab, ac, ..., zzzzzzzzzz

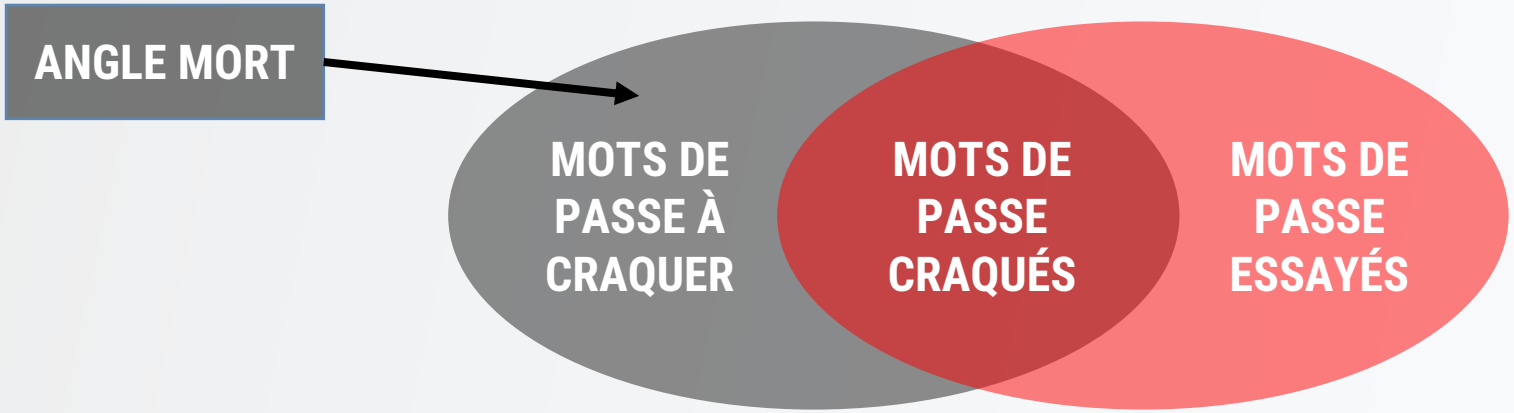
Mais les mots de passe sont devenus plus complexes (majuscules, minuscules, ponctuations)

Les pirates ont donc commencé à faire des attaque «par dictionnaire» (ou par mots de passe probable) : on construit une grande liste de mots susceptibles d'être utilisés comme mot de passe et on ne teste que ceux-là.
Donc par exemple dans la liste il y aura : DarkVador31, Azerty! et MJack5on, mais on n'essaye plus les trucs qui veulent rien dire : GjkMawcPmlq, fdgnasotrmdnbue, ...

LE BIAIS DU CRAQUAGE DE MOTS DE PASSE

Et à force de craquer des mots de passe, on est devenus très bon à ça : très souvent quand l'utilisateur doit mettre une majuscule, ce sera la première lettre du mot de passe. S'il doit mettre un caractère spécial, ce sera souvent «!» et à la fin, etc.

Seulement ... il y a un biais du survivant dans cette approche : nous ne pouvons craquer que les mots de passe que nous avons pensé à mettre dans la liste



LE BIAIS DU CRAQUAGE DE MOTS DE PASSE

Mais ça a changé en 2011 quand Mimikatz est apparu.

Jusqu'ici nous étions à moitié aveugle, nous ne pouvions craquer que les mots de passe que nous étions à même d'imaginer.

Mimikatz est un outil permettant d'aller lire les mots de passe de Windows au moment où ils sont en clair dans la RAM.

Donc là, il n'y a plus de hash à craquer, on voit directement les mots de passe des utilisateur : les étranges, les complexes, les honteux, les inimaginables, ... nous avons ouvert notre second oeil.

On peut désormais adapter nos stratégies de construction de liste de mots de passe probable pour intégrer tous les cas réels qu'on a observés et même les généraliser.

COMPLEX MAIS PREDICTIBLE

Des fois, on ne voit pas trop comment on aurait pu les trouver :

```
ASTAR\sophos ErlKs8_2zbM
ASTAR\d.soria %%%FasciculePonnnèy
```



COMPLEX MAIS PREDICTIBLE

Des fois, on ne voit pas trop comment on aurait pu les trouver :

```
ASTAR\sophos ErlKs8_2zbM
ASTAR\d.soria %%%FasciculePonnnèy
```

Mais d’autres fois ...

```
ASTAR\j.auchedy j0ACHIM1
ASTAR\s.evret Thailande?
ASTAR\d.audebert Enzo2004!
ASTAR\veeam _V334m@dm
ASTAR\p.azedias Pedr0.com
ASTAR\c.gazedet Novembre2022!
ASTAR\e.gorvant C@l!f0rn!@
ASTAR\q.hortenz Woodstock1969.
ASTAR\a.bardeau Wxcvbn12!
```

COMPLEX MAIS PREDICTIBLE

Des fois, on ne voit pas trop comment on aurait pu les trouver :

```
ASTAR\sophos ErlKs8_2zbM
ASTAR\d.soria %%%Fasciul
```



Mais d’autres fois ...

```
ASTAR\j.auchedy j0ACHIM1
ASTAR\s.evret Thailande?
ASTAR\d.audebert Enzo2004!
ASTAR\veeam _V334m@dm
ASTAR\p.azedias Pedr0.com
ASTAR\c.gazedet Novembre2022!
ASTAR\e.gorvant C@l!f0rn!@
ASTAR\q.hortenz Woodstock1969.
ASTAR\a.bardeau Wxcvbn12!
```

COMPLEX MAIS PREDICTIBLE

Des fois, on ne voit pas trop comment on aurait pu les trouver :

```
ASTAR\sophos ErlKs8_2zbM
ASTAR\d.soria %%%Fasciculé
```



Mais d’autres fois ...



```
ASTAR\j.auchedy j0ACHIM1
ASTAR\s.evret Thailande?
ASTAR\d.audebert Enzo2004!
ASTAR\veeam _V334m@dm
ASTAR\p.azedias Pedr0.com
ASTAR\c.gazedet Novembre2022!
ASTAR\e.gorvant C@l!f0rn!@
ASTAR\q.hortenz Woodstock1969.
ASTAR\a.bardeau Wxcvbn12!
```

COMPLEX MAIS PREDICTIBLE

Des fois, on ne voit pas trop comment on aurait pu les trouver :

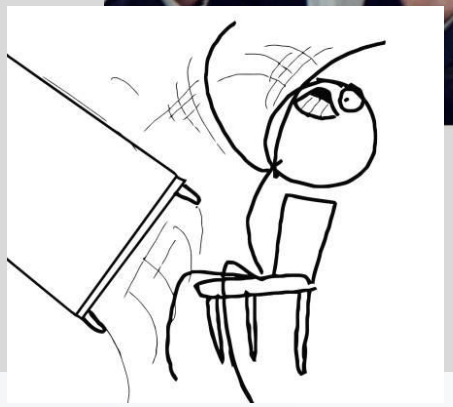
```
ASTAR\sophos ErlKs8_2zbM
ASTAR\d.soria %%%Fasciculé
```



Mais d'autres fois ...

```
ASTAR\j.auchedy j0ACHIM1
ASTAR\s.evret Thailande?
ASTAR\d.audebert Enzo2004!
ASTAR\veeam _V334m@dm
ASTAR\p.azedias Pedr0.com
ASTAR\c.gazedet Novembre2022!
ASTAR\e.gorvant C@l!f0rn!@
ASTAR\q.hortenz Woodstock1969.
ASTAR\a.bardeau Wxcvbn12!
```





COMPLEX MAIS PREDICTIBLE

Ce qu'on observe encore c'est que, bien souvent, le mot de passe tourne autour d'une «racine»

Un racine est un mot **prononçable** que l'on va ensuite embellir ou dégrader en ajoutant des ponctuations, des chiffres, en remplaçant des lettres, etc.

Quand un admin dit à ses utilisateurs : «votre mot de passe doit avoir au moins une minuscule, une majuscule, un chiffre et un caractère spécial»

ce qu'il a en tête c'est un truc de ce genre : kPd3\A#$s9

ce qu'on voit dans les faits, c'est plutôt : Print3mps!

Trouvez la racine, trouver les embellissements communs et le mot de passe sera votre sous peu !

CE QUI EXISTAIT

J’avais l’habitude d’utiliser rsmangler pour dériver des racines :

```
$ cat words.txt
david
soria
lacanau
33
$ rsmangler --file words.txt > words_mangled.txt
$ wc words_mangled.txt
39924  39924 732204 word_mangled.txt
```

Je lui donne : david, soria, lacanau, 33 et il me produit 40 000 mots de passe probables, dont: `david33! - $0r14d@v1d - dsl33 lacanaudavid2009 - davidsoria33lacanau2017 - adminsoria33 - ...`

Pas si mal.

CE QUI EXISTAIT

Pas si mal, mais il y a des limites :

- Si je donne : airbus, avion, a380, a350, 31, toulouse il va me créer : a380aviona350airbus31toulouse2015 ce qui est inutile
- À partir de 4 ou 5 mots, ça devient incalculable
- Je pouvais obtenir david33!, David33 et d@v!d33 mais pas D@vid33! (il ne combine pas ses règles)

En observant de plus en plus de mots de passe réels via Mimikatz, j'ai réalisé que beaucoup n'auraient pas pu être générés par rsmangler (même avec la bonne racine).

CE QUI EXISTAIT

Il y avait aussi les attaques par masque avec hashcat :

- Si je donne le masque David?1?1?s, il va produire toutes les combinaisons depuis David00_ jusqu'à David99$
- On peut créer des masques personalisés : par exemple si je pense que les utilisateurs prennent leur matricule (6 caractères composés de lettres majuscules et de chiffres) comme mot de passe, je peux tenter -1 '?u?d' '?1?1?1?1?1?1'

C'est très polyvalent

Mais en même temps c'est parfois super alambiqué de représenter des habitudes courantes (comme le leet : remplacer E par 3, A par 4, T par 7, etc.)

BON ... ALLEZ JE M'Y COLLE

Je me suis codé un script python MikeThe Mangler.py qui applique précisément les stratégies dont j'ai besoin, et **surtout**, qui les combine :

- Les variations de majuscules : david David DAVID dAVID
- Tenter des surnoms : nicolas -> nini, nic, nico, ncls
- Le l33t (jusqu'à 2 substitutions) : nicolas -> n1colas, n1col4s, nicol4s, ...
- Si je donne plusieurs racines, elles sont combinées deux à deux (mais pas plus) : david,soria,lacanau -> Davidsoria, DavidSoria, DavidLacanau, ~~DavidLacanauSoria~~
- Ajout de prefixes et suffixes : david -> DavidAdm, david@dm, pass_david, ...
- Ajout de chiffres de type data d'anniversaires : david -> David88, David1307!, ~~David3420~~
- Je peux même lui demander de ne retenir que les mots de passe qui font au moins X caractères. Dans les entreprises, il y a souvent une longueur minimale requise, inutile de perdre du temps à tester David88 si c'est 8 caractères minimum

BON ... ALLEZ JE M'Y COLLE

Et ensuite j'ai utilisé ce script pour dériver des listes de racines :

- Les lieux : T4nz4ni4, Toulouse31, ...
- Les noms et prénoms (avec date de naissance) : Zoé2012!, Paul0408, ...
- Les dates : Novembre2019! Summer2k20, ...
- Les noms d'entreprise : Airbus31!, EDF_2k15, SocGen!, ...
- Les noms d'utilisateur : V33am, Conference$, T3st2009, 1mpre$$ion, ...

Parfois je m'en sers aussi pour attaquer les clés WiFI :

- astar,wifi,wireless,invité -> WiFi4st4r!, WIFI_INVITE86!, astarinvité2k15, ...

BON ... ALLEZ JE M'Y COLLE

Et ensuite j'ai utilisé ce script pour dériver des listes de racines :

- Les lieux : T4nz4ni4, Toulouse31, ...
- Les noms et prénoms (avec date de naissance) : Zoé2012!, Paul0408, ...
- Les dates : Novembre2019! Summer2k20, ...
- Les noms d'entreprise : Airbus31!, EDF_2k15, SocGen!, ...
- Les noms d'utilisateur : V33am, Conference$, T3st2009, 1mpre$$ion, ...

Parfois je m'en sers aussi pour attaquer les clés WiFI :

- astar,wifi,wireless,invité -> WiFi4st4r!, WIFI_INVITE86!, astarinvité2k15, ...



BON … ALLEZ JE M’Y COLLE

Les listes font entre 30 MO and 30 GO et se génèrent en quelques secondes / minutes (même pas besoin de les stocker si je veux gagner de l’espace.

Voici quelque mot de passe que j’ai cassé par ce moyen (j’ai changé les noms d’entreprise) :

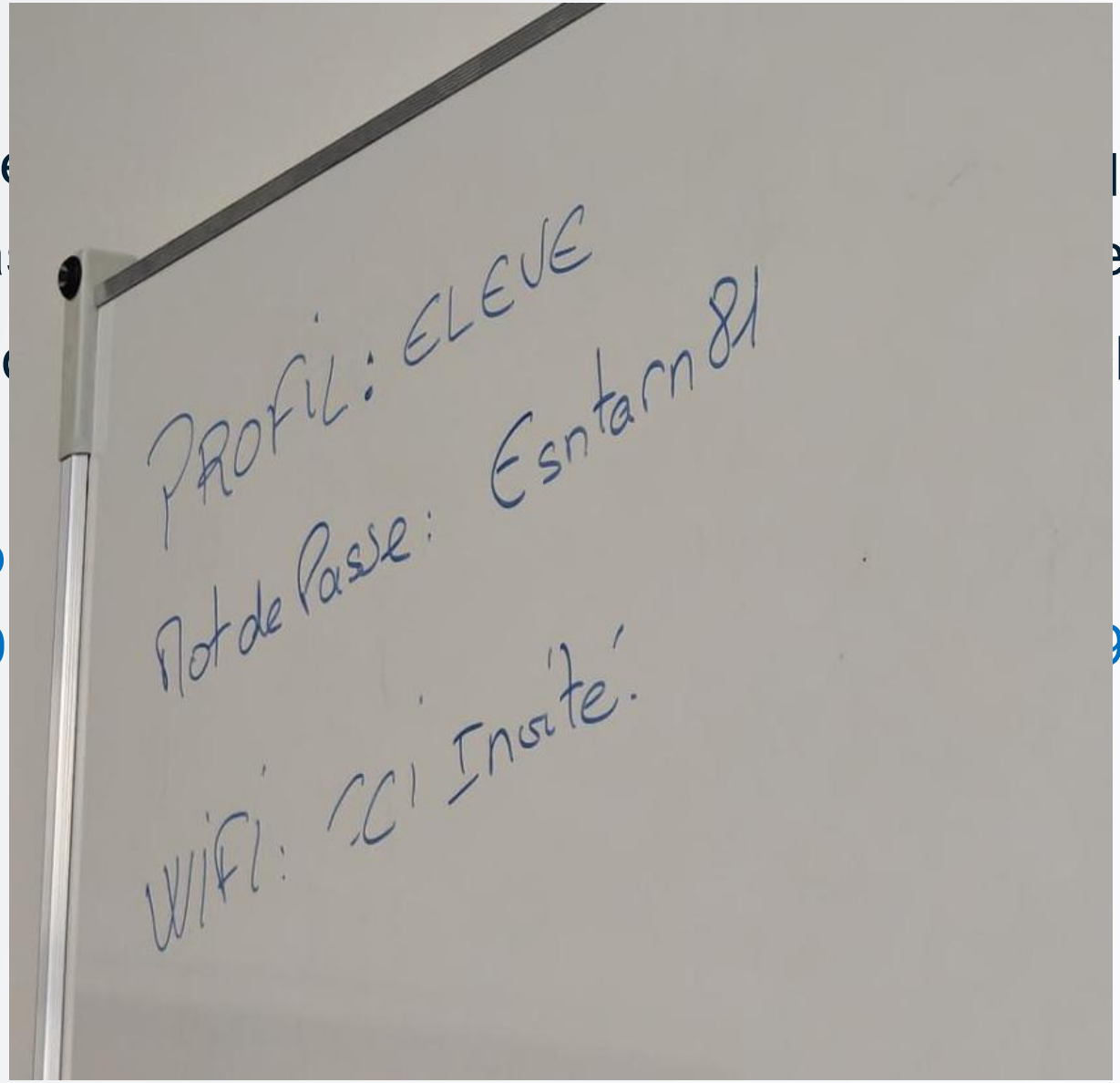
`Lulu2509 - Cho-2000 - Emma1307! - T0ulouse31! - Rostov07 - Avril2021$ - Octavia1 - aURELIA0205 - Madinna972 - 28120107 - July2019`

BON ... ALLEZ JE M'Y COLLE

Les listes font entre ques secondes / minutes (même pas e l'espace.

Voici quelque mot hangé les noms d'entreprise) :

`Lulu2509 - Cho - Rostov07 - Avril2021$ - O 972 - 28120107 - July2019`




UN JOUR TU FERAS PERDRE DES MILLIONS À LA BOITE DE PAPA
EMMA1307

CREDENTIAL STUFFING

MAIS ON PEUT AUSSI FAIRE PLUS MALIN

Depuis que les mots de passe sont lisibles directement (via Mimikatz) et après des années à voler les bases de données des sites (breach) et à casser des hash, les pirates ont créé des immenses listes de mots de passe connus (leak).

- «Connus» ça ne veut pas dire qu'ils sont simples
- Ça ne veut pas dire qu'il sont évident ou prédictibles
- Ça veut dire qu'au moins quelqu'un, quelque part, l'a utilisé une fois

Les pirates s'échangent et se revendent ces immenses listes jusqu'à constituer une base de plusieurs TO et l'utilise pour casser les hash.

Ils continuent d'alimenter cette base chaque fois qu'ils piratent un nouveau site.

MAIS ON PEUT AUSSI FAIRE PLUS MALIN

Donc là il n'y a plus de stratégie de construction de mots de passe probables.

Seulement une approche « quelqu'un l'a déjà utilisé une fois, donc peut être que celui dont j'essaye de casser le hash l'a utilisé aussi »

Ainsi, vous pourriez avoir choisi un mot de passe robuste et imprédictible ($$Garguantua\\77) mais PADBOL quelqu'un d'autre dans le monde avait choisi le même en s'inscrivant sur le site Domino's Pizza et il se l'est fait voler quand Domino's Pizza s'est fait piraté en 2014

Quand les pirates utilisent des bases de mots de passe connus pour tenter de craquer le votre, on appelle ça du **credential stuffing**.

Ça sert notamment beaucoup pour chercher si le même utilisateur réemploie le même couple mail/password sur d'autres sites que celui compromis.



Domino's Data Breach

What Happened

In June 2014, Domino's Pizza in France and Belgium was hacked by a group going by the name "Rex Mundi" and their customer data held to ransom. Domino's refused to pay the ransom and six months later, the attackers released the data along with troves of other hacked accounts. Amongst the customer data was passwords stored with a weak MD5 hashing algorithm and no salt.

Breach Overview

- Affected Accounts: **648.2 thousand**
- Breach Occurred: **June 2014**
- Added to HIBP: **4 Jan 2015**

Compromised Data

- Email addresses
- Passwords
- Physical addresses
- Names
- Phone numbers

MAIS ON PEUT AUSSI FAIRE PLUS MALIN

Donc là il n'y a plus de stratégie de construction de mots de passe probables.

Seulement une approche « quelqu'un l'a déjà utilisé une fois, donc peut être que celui dont j'essaye de casser le hash l'a utilisé aussi »

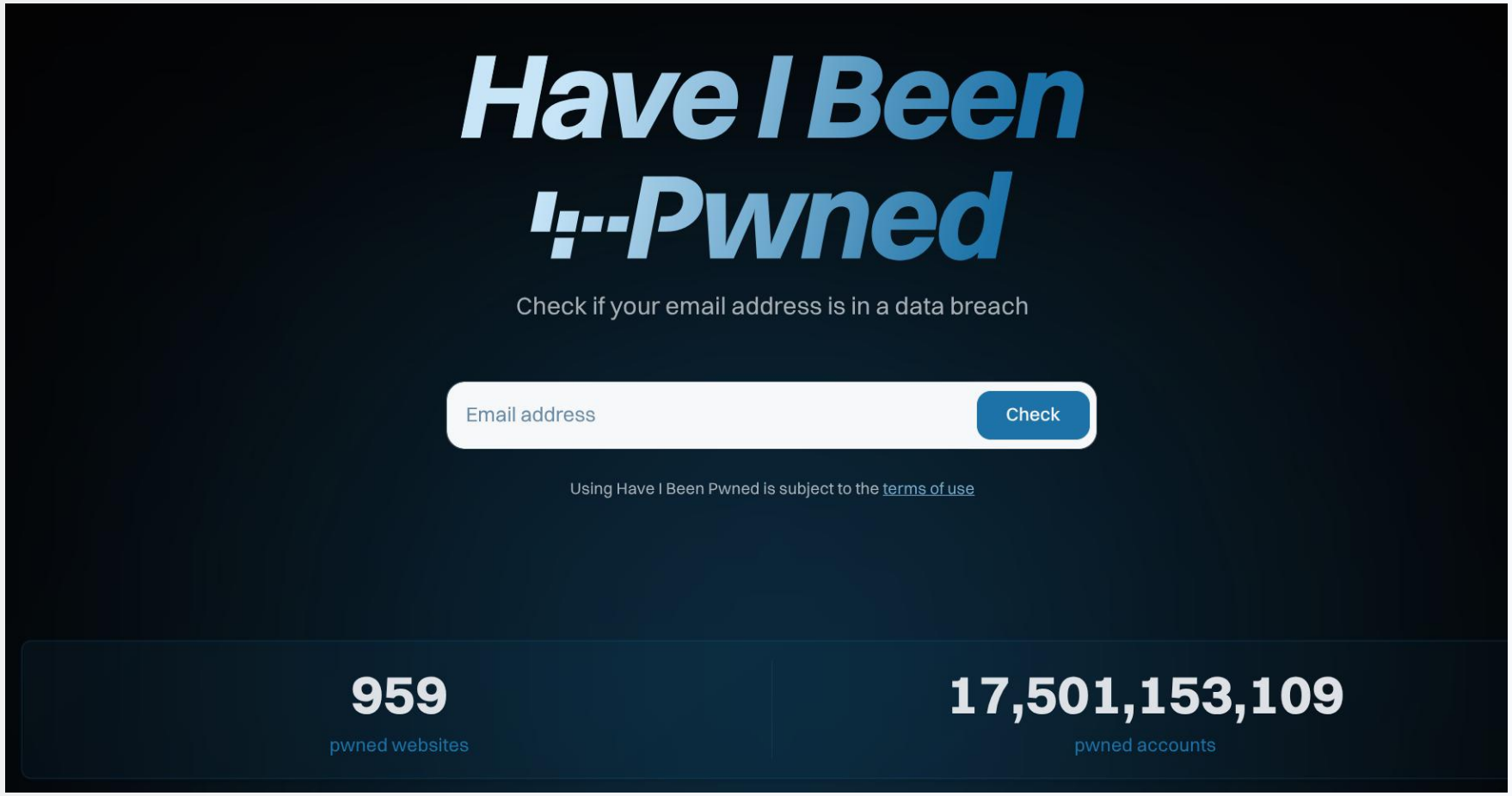
Ainsi, vous pourriez avoir choisi un mot de passe robuste et imprédictible ($$Garguantua\\77) mais PADBOL quelqu'un d'autre dans le monde avait choisi le même en s'inscrivant sur le site Domino's Pizza et il se l'est fait voler quand Domino's Pizza s'est fait piraté en 2014.

Quand les pirates utilisent des bases de mots de passe connus pour tenter de craquer le votre, on appelle ça du **credential stuffing**.

Ça sert notamment beaucoup pour chercher si le même utilisateur réemploie le même couple mail/password sur d'autres sites que celui compromis.

EST-CE QUE ÇA VOUS CONCERNE

Il y a un site très connu pour vérifier si votre mot de passe ou votre adresse email font parties de ces grandes bases de comptes volés :



EST-CE QUE ÇA VOUS CONCERNE

Ce site propose une API à laquelle on peut vérifier de manière confidentielle si un hash donné fait partie de la base des mots de passe volés.

Du coup, on n'a même pas besoin d'essayer de craquer les hash des utilisateurs. On exporte les hash de leurs mots de passe et quand l'un d'eux se retrouve dans la liste de HaveIBennPwned, c'est qu'il est vulnérable.

J'ai codé un petit outil qui permet de vérifier ça :

```
$ python3 LennieTheLeaker.py HASH_FILE
[+] Hash leaked 16484 time(s): 720D0E...B1EE used by {'astar\marc-
antoine', 'astar\franck', 'astar\paul'}
[+] Hash leaked 283 time(s): 37E8A2...67E1BB4A553 used by {astar\odile'}
[+] Hash leaked 2 time(s): EB61B4...CD970 used by {'astar\jeanmich'}
```

MAIS C’EST QUI LE MEC AU DÉPART ?

MAIS C’EST QUI LE MEC AU DÉPART

Casser des hash c’est bien gentil, mais une fois que j’ai compromis le compte, ce qui m’intéresse c’est de savoir ce qu’il a comme accès.

Si c’est un admin du domaine, c’est le graal, mais c’est si un compte désactivé, ça ne me sert à rien.

Avant, dès que je compromettais un nouveau compte, je faisais une requête à l’annuaire pour savoir les droits qu’il possède. Sauf que quand le mot de passe est craqué 7 jours après la fin de la mission, je ne peux plus.

Donc j’ai fait un outil qui récupère la base des utilisateurs (avec leurs groupes) et qui me permet directement de connaître la criticité de tous les comptes que j’ai compromis

MAIS C'EST QUI LE MEC AU DÉPART

Donc j'ai fait un outil LesterTheLooter.py qui va chercher les infos sur les comptes dont le mot de passe est cassable.

En entrée il a besoin de la liste des comptes compromis et de l'export des utilisateurs du domaine (fichier domain_user.grep de ldapdomaindump) :

```
$ python3 LesterTheLooter.py john_result.txt domain_user.grep
[*] Importing john result and domain info
[*] Exporting data to users_compromised.csv and group_compromised.csv
[+]  disabled  domain admins       admgba          LoveWine4Ever
[+]  enabled   likely admin        j.snow          knower0fTh1ngs
[+]  enabled   enterprise admins   l.finger        S4nsa<3
[+]  enabled   domain admins       adm.cla         JaimeJaime
[+]  disabled  account operators   t.lannister     K1ng'sHand
```

RESTITUER

VOILA LES MOTS DE PASSE TROUVÉS

À la fin d'un audit, c'était toujours apprécié de donner au client la liste des mots de passe compromis ("Wow" effect à pas cher).

Mais c'est encore plus classe de donner des statistiques : les motifs les plus communs, les mots de passe les plus utilisés, etc.

Avant j'utilisais pipal :

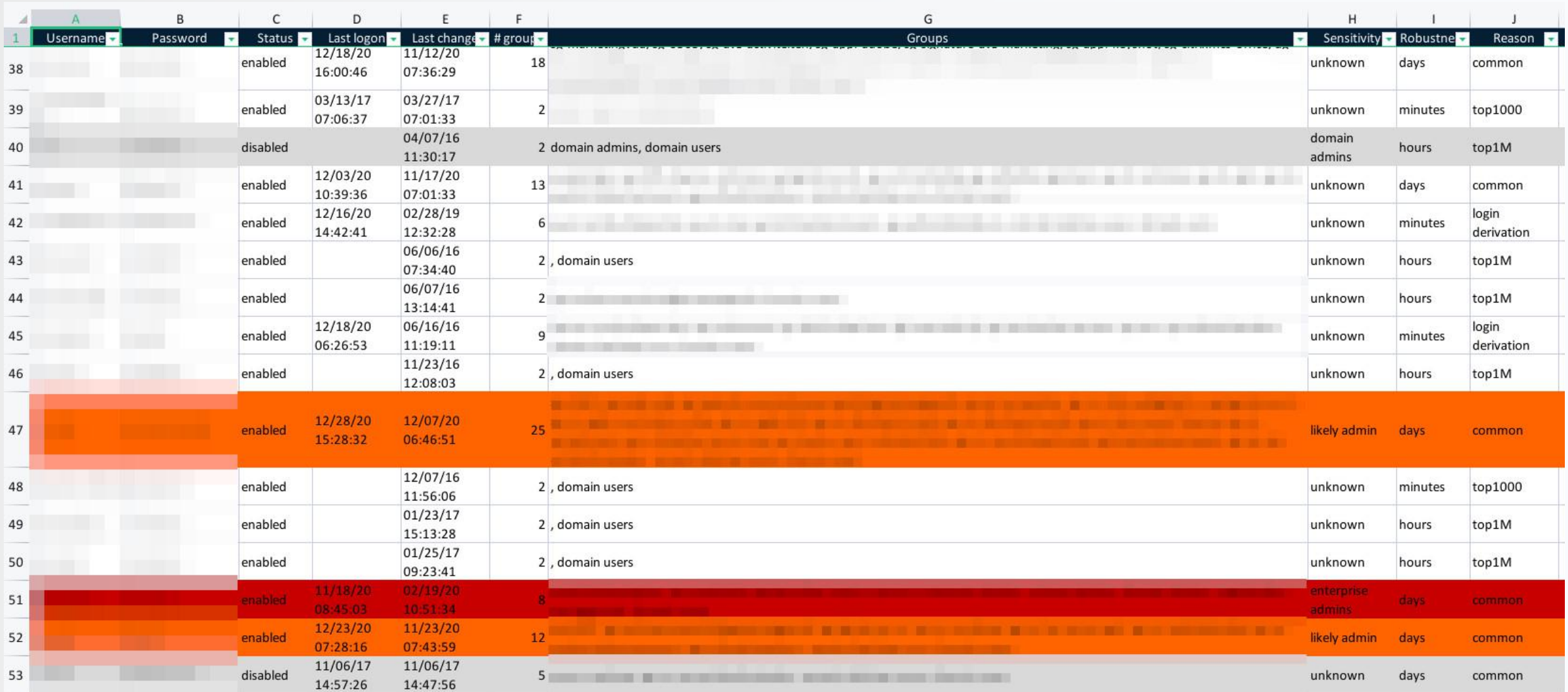
```
$ pipal passwords_cracked.txt
Total entries = 1360                One to six characters = 17 (1.25%)
Total unique entries = 121          One to eight characters = 1297 (95.37'%)
                                    More than eight characters = 63 (4.63%)
Top 10 passwords
P@ssword = 1236 (90.88%)            First capital last symbol = 16 (1.18%)
1234Abcd= 3 (0.22%)                 First capital last number = 106 (7.79%)
...                                 ...
```

VOILA LES MOTS DE PASSE TROUVÉS

Mais c’était moche, donc j’ai rajouté une option --xlsx à LesterTheLooter.py

VOILA LES MOTS DE PASSE TROUVÉS

Mais c'était moche, donc j'ai rajouté une option --xlsx à LesterTheLooter.py



	A	B	C	D	E	F	G	H	I	J
1	Username	Password	Status	Last logon	Last change	# group	Groups	Sensitivity	Robustne	Reason
38			enabled	12/18/20 16:00:46	11/12/20 07:36:29	18		unknown	days	common
39			enabled	03/13/17 07:06:37	03/27/17 07:01:33	2		unknown	minutes	top1000
40			disabled		04/07/16 11:30:17	2	domain admins, domain users	domain admins	hours	top1M
41			enabled	12/03/20 10:39:36	11/17/20 07:01:33	13		unknown	days	common
42			enabled	12/16/20 14:42:41	02/28/19 12:32:28	6		unknown	minutes	login derivation
43			enabled		06/06/16 07:34:40	2	, domain users	unknown	hours	top1M
44			enabled		06/07/16 13:14:41	2		unknown	hours	top1M
45			enabled	12/18/20 06:26:53	06/16/16 11:19:11	9		unknown	minutes	login derivation
46			enabled		11/23/16 12:08:03	2	, domain users	unknown	hours	top1M
47			enabled	12/28/20 15:28:32	12/07/20 06:46:51	25		likely admin	days	common
48			enabled		12/07/16 11:56:06	2	, domain users	unknown	minutes	top1000
49			enabled		01/23/17 15:13:28	2	, domain users	unknown	hours	top1M
50			enabled		01/25/17 09:23:41	2	, domain users	unknown	hours	top1M
51			enabled	11/18/20 08:45:03	02/19/20 10:51:34	8		enterprise admins	days	common
52			enabled	12/23/20 07:28:16	11/23/20 07:43:59	12		likely admin	days	common
53			disabled	11/06/17 14:57:26	11/06/17 14:47:56	5		unknown	days	common

VOILA LES MOTS DE PASSE TROUVÉS

L'onglet Utilisateur montre :

Name	Passwd	Status	lastlogon	lastchange	#groups	groups	sensitive	robustness	Reason
admdso	Astar2021	enabled	2021-04-06	2021-01-03	12	domain admins, ...	admin	minutes	company
kcobain	N!rv@na	disabled	1994-04-05	1994-01-12	2	27 club, ...	unknown	hours	common

L'onglet Groupes montre :

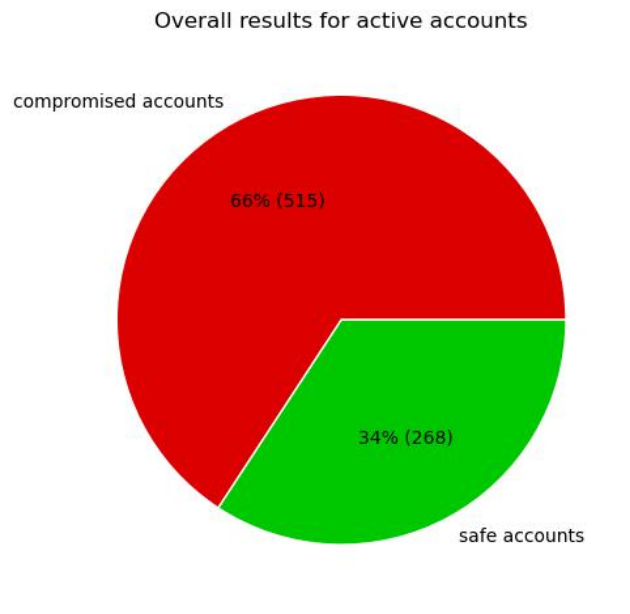
Name	members cracked	Sensitive	robustness	enabled members	disabled members
admins du domaine	3	admin	seconds	admdso, admqpo	adminbackup
club des 27	1	unknown	years		kcobain

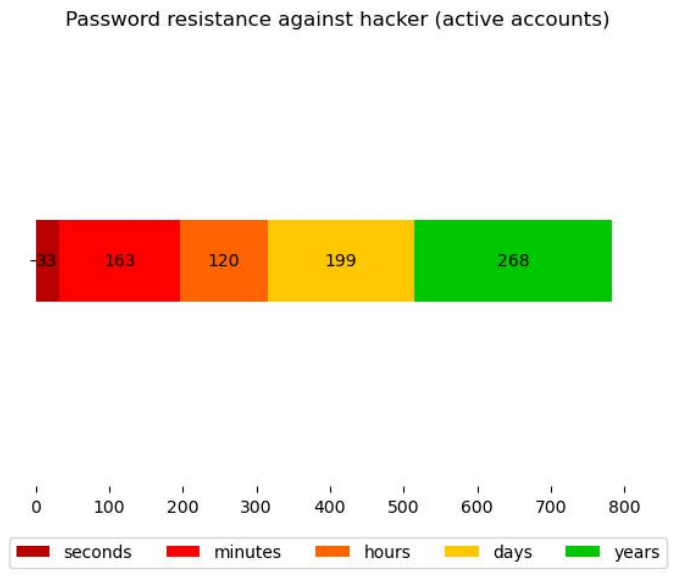
VOILA LES MOTS DE PASSE TROUVÉS

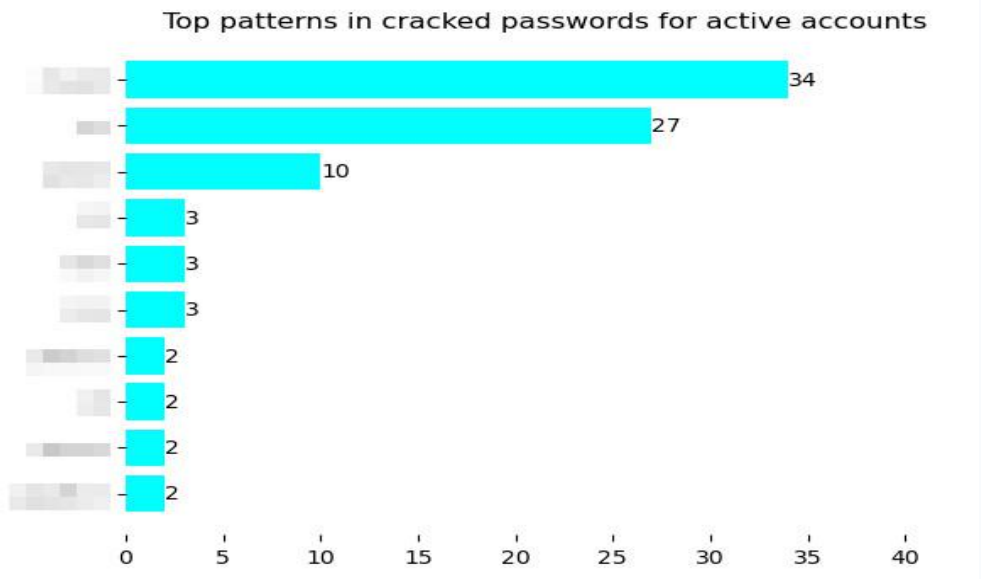
Un tableur c’est bien, mais des visuels c’est mieux.

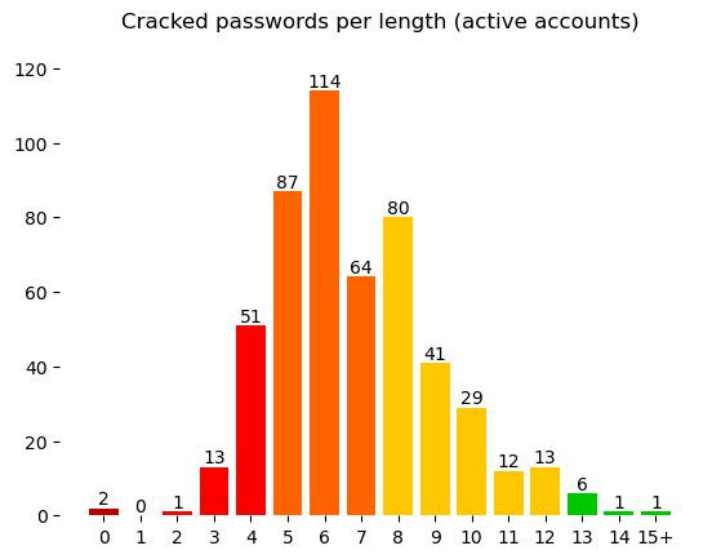
C’était pénible de les générer à chaque fois depuis le tableur, donc maintenant c’est automatique.

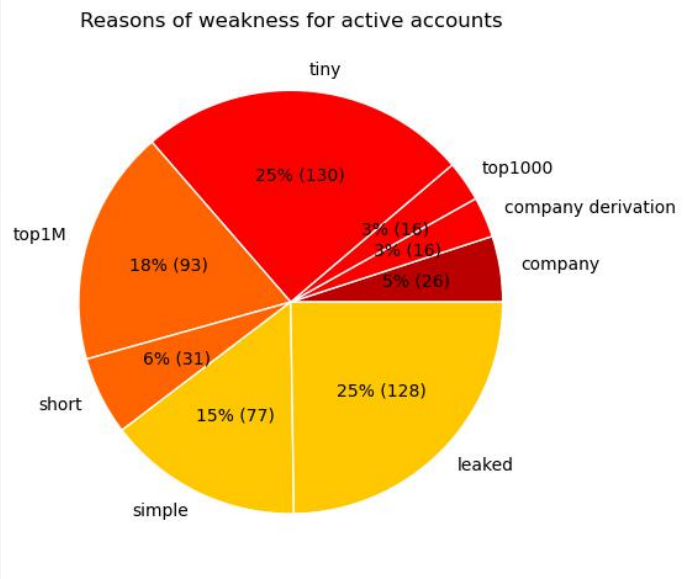
VOILA LES MOTS DE PASSE TROUVÉS

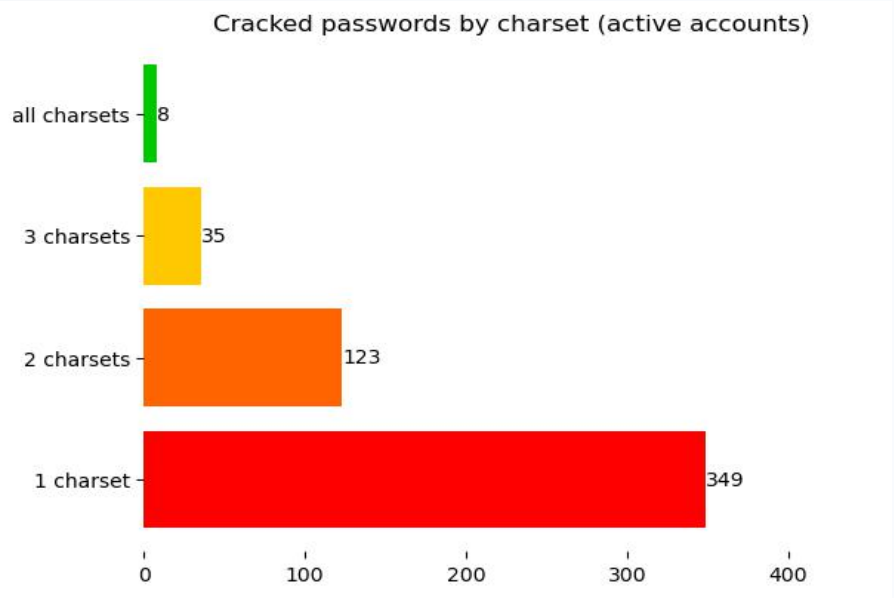












DE QUOI A T-ON BESOIN ?

PRÉREQUIS

Pour utiliser l'outil, il faut lui donner deux fichiers : la base des utilisateurs du domaine via l'outil ldapdomaindump : le fichier domain_user.grep

Le deuxième c'est la base des mots de passe de votre domaine. Vous pouvez l'obtenir depuis les fichiers NTDS.dit et SYSTEM sur votre contrôleur de domaine.

Sauf que maintenant les EDR gueulent quand vous voulez l'exraire avec la commande :

```
$ cmd.exe /c ntdsutil "ac i ntds" "ifm" "create full c:\temp\robb" q q
```

Donc il faudra peut être aller le chercher directement dans la sauvegarde du serveur (dans C:\Windows\ntds.dit et C:\Windows\System32\config\SYSTEM)

Attention c'est un fichier critique qui ne doit pas tomber entre de mauvaises mains (ne le laissez pas trainer)

PRÉREQUIS

Depuis le fichier NTDS.dit on peut extraire les utilisateurs et leurs hash avec l'outil impacket :

```
$ impacket-secretsdump -system SYSTEM -ntds ntds.dit LOCAL > DOMAIN_HASHES.txt
```

J'ai aussi écrit un petit script qui va tout chercher par lui-même en lui fournissant un compte admin du domaine (mais ça peut foirer à cause de l'EDR) :

```
$ ./RobbTheRobber.py --user admdso --password poulpe --domain astar.org
```

RECAP

LAZY AT WILL

Pour un non spcialiste, on peut lister les comptes vulnérables du domaine avec 3 commandes :

```
$ ./RobbTheRobber.py --user admdso --password poulpe --domain astar.org
$ ./LennieTheLeaker.py DOMAIN_HASHES.txt
$ ./LesterTheLooter.py --xlsx JOHN_RESULT.txt USERS_INFO.txt
```

En ayant installé 3 dépendances python :

- ldapdomaindump
- impacket
- click

LAZY GUY MADES LAZY NAME

Comme l'outil me servait à la base à générer les stats sur les mots de passe que j'avais compromis, je l'ai appelé Lestat



MERCI POUR VOTRE ATTENTION

URL du projet : https://github.com/astar-security/Lestat